It can be observed that the code is written based on the contract of the interface. It is always a good idea for consumers to refer only to the interfaces, not the providers. This practice is popular as 'coding against the interfaces'.

Does this code solve the problem of tight coupling? Not completely. The first line still has an explicit reference to the provider. That must go! However, before addressing it, let's take a close look at *InMemoryUserRepository*.

Singleton

The current implementation of *InMemoryUserRepository* offers a constructor that is accessible publicly. It implies that consumers can use the *new* operator to create any number of instances of it. That can create problems!

For example, the *Application* creates an instance of *InMemoryUserRepository* to add user entries. Unknowingly, it can create one more instance of it in another part of the code, to find the user entries. However, the map of the second instance has nothing to do with the map of the first instance. Hence, the *Application* fails to find any user entries in the second instance. The point is that classes like *InMemoryUserRepository* must be created only once across the application. In other words, the *InMemoryUserRepository* must be a 'singleton'.

How do you convert the *InMemoryUserRepository* as a singleton so that it is possible to instantiate it only once in a JVM?

The first step is to make the constructor *private*, to prevent the consumers from instantiating the objects using the *new* operator.

```
private InMemoryUserRepository() { ... }
```

The second step is to introduce a new method, say *getInstance()*, for creating and returning an instance of *InMemoryUserRepository* in a controlled way. The *getInstance()* method must be available at the class level so that the consumers can directly invoke it. This means, *getInstance()* must be *static*.

```
public static InMemoryUserRepository getInstance() {...}
```

With the presence of the above method, the *Application* can get an instance as follows:

```
UserRepository repo = InMemoryUserRepository.getInstance();
```

But how can the *getInstance()* method ensure that it creates and returns only one instance of *InMemoryUserRepository*? Simple! It must create an instance in the beginning and keep returning the same instance every time. The following code does the job!

```
private static InMemoryUserRepository INSTANCE = null;

public static InMemoryUserRepository getInstance() {
     if (INSTANCE == null)
       INSTANCE = new InMemoryUserRepository();
     return INSTANCE;
}
```

By designing the *InMemoryUserRepository* as a singleton, we ensure that the consumers share the one and only instance of the *InMemoryUserRepository*, no matter how many times the method *getInstance()* is called. Take a look at this code:

```
UserRepository first = InMemoryUserRepository.getInstance();
UserRepository second = InMemoryUserRepository.getInstance();
first.add(9731423166L, "Krishna");
String name = second.find(9731423166L);
```

The *first* and *second* are exactly the same objects. This works perfectly. We can follow the same pattern to convert any class as a singleton.

Factory

We thought that tight coupling is a problem when the *Application* wants to choose between *InMemoryUserRepository* and *PersistentUserRepository*. But the tight coupling can be a problem with just *InMemoryUserRepository* also. Let's see how!

Earlier, the *Application* gets the object of *InMemoryUserRepository* as follows:

```
UserRepository repo = new InMemoryUserRepository();
```

But once *InMemoryUserRepository* is refactored as a singleton, the *Application* changes like this:

```
UserRepository repo = InMemoryUserRepository.getInstance();
```

Just because of an internal change in the *InMemoryUserRepository*, the code of the *Application* is also forced to change. This proves again that tight coupling is bad.

How else can the consumer and provider be connected? Like the way it happens in the real world, instead of the consumers creating the providers, they must go to a factory and ask for a provider of a contract. After all, the object-oriented design mimics the real world! The factory must take the responsibility of creating and supplying the provider to the consumers. The *UserRepositoryFactory* does the job nicely.

```
package com.glarimy.ums;
public class UserRepositoryFactory {
     public static UserRepository get() {
```